Event Planner

Dokumentacja Funkcji i Endpointów API

Wersja 1.0 | Luty 2026

---

1. Backend - Kontrolery

> `loginController(req, res)`

Obsługuje logowanie użytkownika. Weryfikuje dane logowania, generuje token JWT i zwraca odpowiedź.

Parametry:

> `req.body.login` (string) - Email lub nazwa użytkownika

> `req.body.password` (string) - Hasło użytkownika

Walidacja:

- Login i hasło są wymagane
- Sprawdzenie czy użytkownik istnieje w bazie
- Porównanie hasła z zahashowaną wersją (bcrypt)

> **Zwraca:** Obiekt z tokenem JWT i danymi użytkownika lub błędem

> `registerController(req, res)`

Rejestruje nowego użytkownika. Haszuje hasło, sprawdza duplikaty i wstawia do bazy danych.

Parametry:

> `req.body.login` (string) - Nazwa użytkownika do rejestracji

> `req.body.password` (string) - Hasło użytkownika

Proces:

- Haszowanie hasła z bcrypt (10 rounds)
- Sprawdzenie dupliwatów (ER_DUP_ENTRY)
- INSERT do tabeli accounts

> **Zwraca:** Komunikat sukcesu lub błędu (duplikat, błąd serwera)

---

2. Frontend - Komponenty

> `ProtectedRoute({ children })`

Komponent wyższego rzędu (HOC) chroniący trasy wymagające autentykacji. Sprawdza obecność tokenu JWT w localStorage.

Parametry:

> `children` (JSX) - Komponenty którymi należy bronić

Logika:

- Pobierz token z localStorage
- Jeśli brak tokenu: Navigate do /login
- Jeśli token istnieje: Renderuj children

> **Zwraca:** JSX (Route z children lub Navigate do Login)

---

3. API Endpoints

Autentykacja

`POST` `/api/login`

Logowanie użytkownika

Request:

```
{ "login": "jan@example.com", "password": "haslo123" }
```

Response (200):

```
{ "message": "Witaj jan! Logowanie udane.", "token": "eyJhbGc...", "user": { "id": 1,
```

```
"username": "jan" } }
```

POST /api/register

Rejestracja nowego użytkownika

Request:

```
{ "login": "jan@example.com", "password": "haslo123" }
```

Response (200):

```
{ "message": "Użytkownik zarejestrowany pomyślnie" }
```

Response (400):

```
{ "message": "Użytkownik o tej nazwie już istnieje" }
```

Eventy

GET /api/events

Pobranie wszystkich dostępnych eventów

Response (200):

```
[ { "id": 1, "title": "Koncert Rockowy", "date": "2026-03-15", "location": "Warszawa",
"price": 150 } ]
```

POST /api/events

Dodanie nowego eventu (wymaga JWT)

Headers:

```
Authorization: Bearer {token}
```

Ulubione

GET /api/favorites

Moje ulubione eventy (wymaga JWT)

`POST` `/api/favorites/:eventId`

Dodanie eventu do ulubionych

`DELETE` `/api/favorites/:eventId`

Usunięcie z ulubionych

Bilety i Rezerwacje

`GET` `/api/user/events`

Moje rezerwacje (wymaga JWT)

`POST` `/api/tickets`

Rezerwacja biletu na event

Request:

```
{ "eventId": 1, "quantity": 2 }
```

---

4. Bezpieczeństwo

Hasła: Haszowane algorytmem bcrypt (10 rounds). Nigdy nie przechowywane w czystym tekście.

Autentykacja: Tokeny JWT przechowywane w localStorage. Wysyłane w nagłówku Authorization.

CORS: Konfiguracja uniemożliwia dostęp z nieautoryzowanych źródeł.

5. Struktura Bazy Danych

Tabela	Kolumny	Przeznaczenie
accounts	id, username, password, created_at	Użytkownicy i autentykacja

Tabela	Kolumny	Przeznaczenie
events	id, title, date, location, price, description	Katalog eventów
favorites	id, user_id, event_id	Ulubione eventy użytkownika
tickets	id, user_id, event_id, quantity, booking_date	Rezerwacje i bilety

6. Przepływ Aplikacji

Logowanie

1. Użytkownik wypisuje login/hasło w Login.jsx
2. Frontend wysyła POST /api/login
3. Backend waliduje i generuje JWT
4. Token zapisywany w localStorage
5. Navigate do Home (ProtectedRoute sprawdza token)

Rezerwacja Biletu

1. Użytkownik kliką "Book" na EventCard.jsx
2. Wysłanie POST /api/tickets z eventId
3. Backend tworzy wpis w tabeli tickets
4. Bilet pojawia się w Tickets.jsx
5. Generowanie PDF z kodem QR (TicketPDF.jsx)